

C# Programming Essentials

A Practical Guide for Software Engineering Students

Jam Savage

Prepared for UICT Students

Table of Contents

Preface

- Course Description
- Course Objectives
- Learning Outcomes
- How to use this book
- Recommended Textbooks

Introduction to C#

- Explain the C# language features
- Define and describe Visual Studio 2012
- Explain the elements of Microsoft Visual Studio 2012 IDE
- Try It Yourself
- Chapter Summary

Variables and Data Types

- Define and describe variables and data types in C#
- Explain comments
- List the keywords and escape sequences
- Explain input and output
- Try It Yourself
- Chapter Summary

Statements and Operators

- Define and describe statements and expressions
- Explain the types of operators
- Explain the process of performing data conversions in C#
- Try It Yourself
- Chapter Summary

C# Programming Constructs

- Explain selection constructs
- Describe loop constructs
- Explain jump statements in C#
- Try It Yourself
- Chapter Summary

Classes and Methods

- Explain classes and objects
- Define and describe methods
- List the access modifiers
- Explain method overloading
- Define and describe constructors and destructors
- Try It Yourself
- Chapter Summary

Inheritance and Polymorphism

- Define and describe inheritance
- Explain method overriding
- Define and describe sealed classes
- Explain polymorphism
- Try It Yourself
- Chapter Summary

Abstract Classes and Interfaces

- Define and describe abstract classes
- Explain interfaces
- Compare abstract classes and interfaces
- Try It Yourself
- Chapter Summary

Properties and Indexers

- Define properties in C#

- Explain properties, fields, and methods

- Explain indexers

- Try It Yourself

- Chapter Summary

Generics and Iterators

- Define and describe generics

- Explain creating and using generics

- Explain iterators

- Try It Yourself

- Chapter Summary

Exception Handling

- Define and describe exceptions

- Explain the process of throwing and catching exceptions

- Explain nested try and multiple catch blocks

- Define and describe custom exceptions

- Try It Yourself

- Chapter Summary

Conclusion

Preface

Welcome to **C# Programming Essentials**. My name is Jam Savage, and I have taught application programming at UICT for many years. I wrote this book because I noticed that many beginner students understand programming concepts much faster when they are tied to **real-life situations** — a school system, a bank account, a shop till, a phone contact list — rather than abstract, made-up examples.

This book follows the official UICT course outline for **Application Programming with C#** exactly, unit by unit. It is built around the course's stated objectives: understanding object-orientation in C#, using the .NET framework effectively, mastering inheritance and polymorphism, and working confidently with abstract classes and interfaces.

Course Description

This is a course in application programming with an emphasis on software development using C#. The learning activities — classroom, laboratory, and online tasks — are designed to build the knowledge and skills you need to write effective computer programs for real information system applications.

Course Objectives

By studying this book and completing its exercises, you will:

1. Understand the basic concepts of object-oriented programming in C#.
2. Understand how to use modern programming languages more effectively.
3. Understand that C# is an object-oriented programming language developed by Microsoft, intended to be simple, modern, and general-purpose for application development.
4. Master inheritance and polymorphism in C#.
5. Master abstract classes and interfaces in C#.

Learning Outcomes

A student who successfully completes this course will be able to:

1. Apply MS .NET framework application concepts developed by Microsoft.
2. Use XML in C#.NET, specifically ADO.NET and SQL Server.
3. Apply C# basics, objects and types, and inheritance in application development.
4. Develop, implement, and create applications with C#.
5. Develop, implement, and demonstrate component services, threading, remoting, Windows services, and web applications.
6. Apply assemblies and deployment in .NET, and understand mobile application development.

How to use this book

- **Read the explanation, then study the real-life example.** Every concept is tied to something you already understand from daily life — a shop, a school, a bank, a phone — before it is shown in code.
- **Type every example into Visual Studio yourself.** Do not just read code — run it, break it, and fix it.
- **Attempt every “Try It Yourself” task.** These are short and designed to be done in a few minutes.
- **Revise using the chapter summaries** before tests and exams.

Recommended Textbooks

1. *C# Programming: The Ultimate Way to Learn the Fundamentals of the C# Language*.
2. P. J. Deitel and H. M. Deitel, *Visual C# 2008 How to Program*, 3rd Edition, Pearson.
3. Herbert Schildt, *C# 4.0: The Complete Reference*, McGraw-Hill Osborne.

Work hard, practice daily, and you will pass this course with confidence.

— Jam Savage, UICT

Introduction to C#

Real-life analogy for this chapter: Think of building an application like building a house. C# is the *language* the builders speak to each other, Visual Studio is the *workshop* full of tools, and the .NET Framework is the *warehouse* of pre-made materials (bricks, doors, windows) the builders can pick off the shelf instead of making from scratch.

Explain the C# language features

C# (pronounced “C-Sharp”) is a modern, object-oriented programming language developed by Microsoft, first released in 2000 as part of the **.NET initiative**. It was designed by Anders Hejlsberg’s team to combine the power of C++ with the simplicity and productivity of languages like Visual Basic.

Key features of C#:

- **Object-Oriented** — everything is modelled using classes and objects, just like real-world entities. A `BankAccount` class in code mirrors a real bank account: it has a balance (data) and can deposit/withdraw (behaviour).
- **Type-Safe** — the compiler checks that you don’t accidentally mix incompatible data types (e.g., treating a phone number as a mathematical number and multiplying it), which prevents many bugs before the program even runs.
- **Component-Oriented** — C# is designed around building independent, reusable components (like `properties`, `events`, and `assemblies`) — similar to how a phone is built from independent, swappable components: battery, screen, camera.
- **Part of the .NET Framework/Platform** — C# programs run on the **Common Language Runtime (CLR)**, giving them automatic memory management (garbage collection), exception handling, and access to a massive standard library.
- **Modern and general-purpose** — used to build desktop applications, web applications (ASP.NET), mobile apps (Xamarin/.NET MAUI), games (Unity engine), and cloud services (Azure).
- **Structured and rich in built-in functionality** — supports familiar constructs (loops, conditionals) as well as advanced features like generics, LINQ, delegates, and `async/await`.

Real-life example: Imagine a **university student registration system**, used daily at UICT. Every student is represented as an object with data (name, registration number, course) and behaviour (register for a unit, check results). C#’s object-oriented nature makes modelling this system natural:

```
using System;

class Student
{
    // Data (fields) — what a student HAS
    public string Name;
    public string RegistrationNumber;

    // Behaviour (method) — what a student CAN DO
    public void DisplayDetails()
    {
        Console.WriteLine($"Student: {Name}, Reg No: {RegistrationNumber}");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Student s1 = new Student();
        s1.Name = "Nakato Grace";
        s1.RegistrationNumber = "UICT/2024/001";
        s1.DisplayDetails();
    }
}
```

Output:

```
Student: Nakato Grace, Reg No: UICT/2024/001
```

Define and describe Visual Studio 2012

Visual Studio is Microsoft’s official **Integrated Development Environment (IDE)** — a single application that bundles everything a developer needs: a code editor, a compiler, a debugger, and design tools, all in one place. **Visual Studio 2012** was a widely used version in classrooms and industry for building C#/.NET applications (modern courses may use later versions such as Visual Studio 2019/2022 or the lightweight Visual Studio Code, but the core workflow taught here is the same).

Think of Visual Studio like a **fully equipped garage for a mechanic**: instead of hunting for tools all over town (a text editor here, a separate compiler there, a separate debugging tool elsewhere), everything you need is in one place, organised and ready to use.

Key capabilities of Visual Studio:

- **Write code** with intelligent autocomplete (called *IntelliSense*), which suggests class names, method names, and parameters as you type — much like predictive text on a phone.
- **Compile and build** your project into an executable program with a single click or keystroke (F5 or Ctrl+Shift+B).
- **Debug** your program by pausing execution (breakpoints), stepping through code line by line, and inspecting variable values — like a mechanic pausing an engine mid-run to check each part.
- **Design user interfaces** visually, by dragging and dropping buttons, textboxes, and labels onto a form (for Windows Forms or WPF applications).
- **Manage projects and solutions**, organising related files, references, and settings.

Explain the elements of Microsoft Visual Studio 2012 IDE

When you open Visual Studio, you will typically see the following elements:

- **Solution Explorer** — a panel (usually on the right) showing the structure of your project: files, folders, and referenced libraries. Think of it as the *table of contents* for your entire application.
- **Code Editor** — the main central window where you type your C# code, with syntax highlighting (colour-coding keywords, strings, comments) to make code easier to read.
- **Toolbox** — a panel containing draggable UI controls (buttons, labels, textboxes) used when designing graphical forms.
- **Properties Window** — shows and lets you edit the properties (e.g., colour, size, text) of a selected control or file, similar to a settings panel.
- **Output Window** — displays messages from the compiler, such as build progress and error/warning messages, when you build your project.
- **Error List** — lists all compilation errors and warnings in your project, with the file and line number, so you can jump straight to the problem.
- **Menu Bar and Toolbar** — provide quick access to common commands like File > New Project, Build, Debug > Start Debugging (F5), and Debug > Start Without Debugging (Ctrl+F5).
- **Start Page** — the welcome screen shown when Visual Studio opens, with shortcuts to create a new project or open a recent one.

Real-life example — creating your first project:

1. Open Visual Studio and select **File → New → Project**.
2. Choose **Console App (.NET Framework)** and name it `MyFirstApp`.
3. Visual Studio generates a starter file called `Program.cs`, already containing a `Main` method — this is the **entry point** of every C# program (the first code that runs).
4. Type the following inside the `Main` method:

```
Console.WriteLine("Welcome to UICT, home of software engineers!");
Console.ReadKey();
```

5. Press **F5** (or click Start) to build and run the program. A console window pops up showing your message.

Common mistake: Forgetting `Console.ReadKey();` at the end of a console application — without it, the console window flashes and closes immediately before you can read the output.

Try It Yourself

1. Open Visual Studio, create a new **Console App** project called `MyProfile`.
2. Write a program that prints your name, your course, and your favourite subject, each on its own line using `Console.WriteLine`.
3. Locate the Solution Explorer, Toolbox, and Error List panels in your IDE, and identify what each one is for.

Chapter Summary

- C# is a modern, object-oriented, type-safe language developed by Microsoft as part of the .NET initiative.
- C# programs run on the Common Language Runtime (CLR), which provides memory management and a rich standard library.
- Visual Studio is Microsoft's IDE — a single workspace containing an editor, compiler, and debugger.
- Key IDE elements include the Solution Explorer, Code Editor, Toolbox, Properties Window, Output Window, and Error List.
- Every C# console application begins execution in the `Main` method.

Variables and Data Types

Real-life analogy: A variable is like a **labelled container** — think of the labelled jars in a kitchen: one jar labelled “Sugar” holds sugar, another labelled “Salt” holds salt. You cannot pour rice into the jar labelled “Sugar” and still call it sugar. In the same way, a variable has a name (label) and a data type (what kind of “ingredient” it is allowed to hold).

Define and describe variables and data types in C#

A **variable** is a named location in computer memory used to store a value that can change during the execution of a program. Before you can use a variable in C#, you must **declare** it by specifying its **data type** and **name**.

```
dataType variableName = value;
```

Example — modelling a **shop till** that records a sale:

```
string productName = "Sugar 1kg";
double price = 4500.50;
int quantity = 3;
bool isPaid = true;
```

C# is a **statically-typed** language: once you declare a variable with a certain type, it can only ever hold values of that type (or something compatible with it). This is enforced at compile time, catching many errors before the program even runs — like the kitchen jar that only ever holds sugar, no matter what.

Common built-in (primitive) data types in C#

Data Type	Description	Real-life example	Example value
int	Whole numbers	Number of students in class	45
double	Decimal (floating-point) numbers	Price of an item	4500.50
float	Smaller-precision decimal numbers	Weight of a parcel (kg)	2.5f
char	A single character	Grade letter	'A'
bool	True or false	Whether a bill is paid	true
string	A sequence of characters (text)	A customer’s name	"Nakato Grace"
decimal	High-precision decimal, ideal for money	Bank account balance	150000.75m
long	Large whole numbers	National ID number	1234567890123

Real-life example — a school report card:

```
using System;

class Program
{
    static void Main()
    {
        string studentName = "Okello James";
        int age = 16;
        char grade = 'A';
        double averageScore = 87.5;
        bool hasPassed = true;

        Console.WriteLine($"{studentName}, Age: {age}, Grade: {grade}, " +
            $"Average: {averageScore}%, Passed: {hasPassed}");
    }
}
```

Explain comments

Comments are notes in your code that the compiler ignores — they exist purely to help human readers (including your future self) understand what the code is doing. Think of comments like **sticky notes** a shop attendant leaves on a shelf to remind colleagues why items are arranged a certain way — they don't affect the shop's operation, but they help people understand it.

C# supports three comment styles:

```
// Single-line comment: explains the next line of code
int age = 16; // student's age in years

/* Multi-line comment:
   used for longer explanations
   that span several lines */

/// <summary>
/// XML documentation comment: used to generate official documentation,
/// often shown by IntelliSense when other code calls this method.
/// </summary>
void CalculateFees() { }
```

Best practice: Comment on *why* something is done, not just *what* is done — the code itself already shows *what* is happening; a good comment explains the reasoning a reader can't see just from the code.

List the keywords and escape sequences

Keywords are reserved words in C# that have a special, predefined meaning to the compiler. You cannot use a keyword as a variable name. Some common C# keywords:

class, public, private, static, void, int, string, bool, if, else, for, foreach, while, do, switch, case, break, continue, return, new, namespace, using, try, catch, finally, throw, abstract, interface, override, virtual, sealed, base, this, null, true, false.

Escape sequences are special character combinations, starting with a backslash (\), used to represent characters that would otherwise be difficult to type directly inside a string (such as a new line or a quotation mark).

Escape Sequence	Meaning	Real-life example
\n	New line	Printing a receipt with items on separate lines
\t	Tab (horizontal space)	Aligning columns in a printed table
\\	A literal backslash	Printing a Windows file path C:\Users
\"	A literal double-quote	Printing a quoted phrase inside a string
\'	A literal single-quote	Printing an apostrophe inside a char

Real-life example — printing a receipt:

```
Console.WriteLine("UICT CANTEEN RECEIPT\n-----");
Console.WriteLine("Item\tQty\tPrice");
Console.WriteLine("Soda\t2\t2000");
Console.WriteLine("She said, \"Keep the change.\");
Console.WriteLine("File saved at: C:\\Users\\Student\\Documents");
```

Output:

```
UICT CANTEEN RECEIPT
-----
Item    Qty    Price
Soda    2      2000
She said, "Keep the change."
File saved at: C:\Users\Student\Documents
```

Explain input and output

Output means displaying information to the user (usually on the console), and **input** means collecting information typed in by the user while the program is running.

- `Console.WriteLine(...)` — prints text and moves to a new line afterwards.
- `Console.Write(...)` — prints text without moving to a new line.
- `Console.ReadLine()` — reads a full line of text typed by the user, and always returns it as a `string`.
- `Console.ReadKey()` — waits for and reads a single key press (often used to pause the console before it closes).

Because `Console.ReadLine()` always returns a `string`, you must **convert** it if you need a number, using methods like `Convert.ToInt32()`, `Convert.ToDouble()`, or `int.Parse()`.

Real-life example — a simple mobile airtime top-up program:

```
using System;

class Program
{
    static void Main()
    {
        Console.Write("Enter your phone number: ");
        string phoneNumber = Console.ReadLine();

        Console.Write("Enter amount to top up (UGX): ");
        double amount = Convert.ToDouble(Console.ReadLine());

        Console.WriteLine($"You have topped up UGX {amount} onto {phoneNumber}.");
        Console.WriteLine("Thank you for using UIC Telecom!");
    }
}
```

Common mistake: Trying to do arithmetic directly on the result of `Console.ReadLine()` without converting it first — since it is a `string`, `Console.ReadLine() + 5` will not add numbers; it will either fail to compile or behave unexpectedly.

Try It Yourself

1. Write a program that declares variables for a **product name** (`string`), **price** (`double`), and **quantity** (`int`), then prints a formatted receipt line using string interpolation (`$"..."`).
2. Write a program that asks the user to enter their name and age using `Console.ReadLine()`, converts the age to an integer, and prints "Hello [name], next year you will be [age+1] years old."
3. Add at least one single-line comment and one multi-line comment to your program explaining what it does.

Chapter Summary

- A variable is a named container in memory with a fixed data type, declared as `type name = value;`.
- C# is statically typed: a variable's type cannot change after declaration.
- Common data types include `int`, `double`, `float`, `char`, `bool`, `string`, and `decimal`.
- Comments (`//`, `/* */`, `///`) are ignored by the compiler but help human readers understand the code.
- Keywords are reserved words with special meaning and cannot be used as identifiers.
- Escape sequences (`\n`, `\t`, `\\`, `\"`) let you embed special characters inside strings.
- `Console.WriteLine` / `Console.Write` handle output; `Console.ReadLine` / `Console.ReadKey` handle input, with `ReadLine()` always returning a `string` that may need conversion.

Statements and Operators

Real-life analogy: If a C# program is a recipe, then **statements** are the individual instructions (“boil the water”, “add the rice”), and **operators** are the small actions performed on ingredients (“add”, “mix”, “compare the temperature”).

Define and describe statements and expressions

A **statement** is a complete instruction that performs an action. In C#, most statements end with a semicolon (`;`). Examples of statements: declaring a variable, calling a method, or making a decision with `if` .

An **expression** is a piece of code that **evaluates to a value**. Expressions are often used *inside* statements.

```
int total = price * quantity; // "price * quantity" is an EXPRESSION
                             // the whole line is a STATEMENT
```

Real-life example — calculating a taxi fare:

```
double baseFare = 2000; // statement: declaration
double perKmRate = 500; // statement: declaration
double distanceKm = 12; // statement: declaration

double totalFare = baseFare + (perKmRate * distanceKm); // expression evaluated, then assigned

Console.WriteLine($"Total fare: UGX {totalFare}"); // statement: method call
```

Here, `baseFare + (perKmRate * distanceKm)` is an expression that evaluates to `8000` , and the entire line `double totalFare = ...;` is a statement.

Explain the types of operators

C# operators are symbols that perform operations on one or more values (called **operands**).

1. Arithmetic operators — for calculations

Operator	Meaning	Example (school fees)
<code>+</code>	Addition	<code>tuition + functionalFee</code>
<code>-</code>	Subtraction	<code>totalFee - amountPaid</code>
<code>*</code>	Multiplication	<code>pricePerUnit * quantity</code>
<code>/</code>	Division	<code>totalFee / numberOfInstallments</code>
<code>%</code>	Modulus (remainder)	<code>studentCount % groupSize</code>

2. Relational (comparison) operators — for comparing values, always returning `true` or `false`

Operator	Meaning	Example
<code>==</code>	Equal to	<code>age == 18</code>
<code>!=</code>	Not equal to	<code>status != "Paid"</code>
<code>></code>	Greater than	<code>balance > 0</code>
<code><</code>	Less than	<code>score < 50</code>
<code>>=</code>	Greater than or equal to	<code>age >= 18</code>
<code><=</code>	Less than or equal to	<code>stock <= 5</code>

3. Logical operators — for combining true/false conditions

Operator	Meaning	Example (loan approval)
<code>&&</code>	AND (both must be true)	<code>hasJob && creditScore > 600</code>
<code>& \ </code>	OR (at least one must be true)	<code>isStudent & \ isSeniorCitizen</code>

!	NOT (reverses true/false)	!hasOutstandingDebt
---	---------------------------	---------------------

4. Assignment operators — for storing/updating values

Operator	Meaning	Example
=	Assign	balance = 5000;
+=	Add and assign	balance += deposit;
-=	Subtract and assign	balance -= withdrawal;
*=	Multiply and assign	total *= 1.18; (add 18% VAT)
/=	Divide and assign	average /= count;

5. Increment / decrement operators

Operator	Meaning	Example
++	Increase by 1	attendanceCount++;
--	Decrease by 1	stockLevel--;

Real-life example — a bank ATM withdrawal check:

```
using System;

class Program
{
    static void Main()
    {
        double balance = 150000;
        double withdrawalAmount = 50000;
        bool hasActiveAccount = true;

        bool canWithdraw = hasActiveAccount && (withdrawalAmount <= balance) && (withdrawalAmount > 0);

        if (canWithdraw)
        {
            balance -= withdrawalAmount; // subtract and assign
            Console.WriteLine($"Withdrawal successful. New balance: UGX {balance}");
        }
        else
        {
            Console.WriteLine("Withdrawal denied.");
        }
    }
}
```

Explain the process of performing data conversions in C#

Sometimes you need to convert a value from one data type to another — for example, converting text typed by a user into a number, or converting a decimal price into a whole number for display. C# supports two main kinds of conversion:

1. Implicit conversion (automatic, safe)

Happens automatically when converting from a **smaller** type to a **larger, compatible** type — no data is lost, so C# does it for you.

```
int wholeMarks = 85;
double marksWithDecimals = wholeMarks; // int automatically becomes double: 85.0
```

2. Explicit conversion (casting — you must request it)

Required when converting from a **larger** type to a **smaller** type, because data might be lost (e.g., decimal places dropped). You must use a **cast** to tell the compiler you understand the risk.

```
double averageScore = 87.9;
int roundedScore = (int)averageScore; // explicit cast: becomes 87 (decimal part is cut off, not rounded!)
```

3. Using the `Convert` class and `Parse` methods

Most commonly used when converting **strings** (e.g., from user input) into numeric types:

```
string ageInput = "18";
int age = Convert.ToInt32(ageInput); // Convert class
int age2 = int.Parse(ageInput); // Parse method – similar result
double price = Convert.ToDouble("4500.75");
```

Real-life example — currency conversion at a forex bureau:

```
using System;

class Program
{
    static void Main()
    {
        Console.Write("Enter amount in USD: ");
        double usdAmount = Convert.ToDouble(Console.ReadLine());

        double exchangeRate = 3800.0; // UGX per USD
        double ugxAmount = usdAmount * exchangeRate;

        int roundedUgx = (int)Math.Round(ugxAmount); // explicit conversion after rounding

        Console.WriteLine($"UGX {roundedUgx} (rounded from {ugxAmount})");
    }
}
```

Common mistake: Casting `(int)` on a `double` truncates (cuts off) the decimal part instead of rounding it — `(int)87.9` gives `87`, not `88`. If you want proper rounding, use `Math.Round()` first, as shown above.

Try It Yourself

1. Write a program that calculates the total cost of items in a shopping basket (price × quantity for three different items), then applies an 18% VAT using `*=`.
2. Write a program using relational and logical operators to check loan eligibility: a person qualifies if their age is between 21 and 60 (inclusive) **and** their monthly income is above UGX 500,000.
3. Ask the user to enter a temperature in Celsius as text, convert it to a `double`, and convert it to Fahrenheit using the formula $F = C * 9/5 + 32$.

Chapter Summary

- A statement performs an action; an expression evaluates to a value and is often embedded within a statement.
- C# has arithmetic, relational, logical, assignment, and increment/decrement operators, each serving a distinct purpose.
- Implicit conversions happen automatically and safely; explicit conversions (casts) are required when data might be lost.
- The `Convert` class and `Parse` methods are the standard way to convert user input (strings) into numeric types.

C# Programming Constructs

Real-life analogy: Imagine a **traffic officer at a junction**. Selection constructs are like the officer deciding which road to send a car down based on its destination. Loop constructs are like a security guard who checks every single car entering a gate, one after another, until the queue is finished. Jump statements are like a shortcut sign or a “stop, no further checks needed” signal.

Explain selection constructs

Selection constructs allow a program to choose between different paths of execution based on a condition — just as a bank teller decides whether to approve or reject a withdrawal based on the account balance.

The `if` statement

```
double balance = 20000;
double withdrawal = 30000;

if (withdrawal > balance)
{
    Console.WriteLine("Insufficient funds.");
}
```

The `if...else` statement

```
int age = 16;

if (age >= 18)
{
    Console.WriteLine("You are eligible to vote.");
}
else
{
    Console.WriteLine("You are not yet eligible to vote.");
}
```

The `if...else if...else` ladder — for multiple conditions

Real-life example — grading a student’s exam score:

```
int score = 68;

if (score >= 80)
{
    Console.WriteLine("Grade: A");
}
else if (score >= 70)
{
    Console.WriteLine("Grade: B");
}
else if (score >= 60)
{
    Console.WriteLine("Grade: C");
}
else if (score >= 50)
{
    Console.WriteLine("Grade: D");
}
else
{
    Console.WriteLine("Grade: F");
}
```

The `switch` statement — cleaner for many discrete options

Real-life example — a matatu (taxi) fare board based on the day of the week:

```

string day = "Saturday";
double fare;

switch (day)
{
    case "Saturday":
    case "Sunday":
        fare = 3000; // weekend fare
        break;
    case "Monday":
    case "Tuesday":
    case "Wednesday":
    case "Thursday":
    case "Friday":
        fare = 2000; // weekday fare
        break;
    default:
        fare = 0;
        Console.WriteLine("Invalid day.");
        break;
}

Console.WriteLine($"Fare for {day}: UGX {fare}");

```

Common mistake: Forgetting the `break;` statement at the end of each `case` — in C# (unlike some languages), leaving it out is actually a **compile-time error** for non-empty cases, which helps beginners avoid accidental “fall-through” bugs.

Describe loop constructs

Loops repeat a block of code multiple times — just like a teacher marking every single exam script in a pile, one after another, without writing separate code for each script.

The `for` loop — best when you know how many times to repeat

Real-life example — printing 5 receipts:

```

for (int i = 1; i <= 5; i++)
{
    Console.WriteLine($"Printing receipt number {i}...");
}

```

The `while` loop — repeats while a condition remains true (checked before each iteration)

Real-life example — an ATM that keeps asking for a PIN until it is correct or attempts run out:

```

int correctPin = 1234;
int attempts = 0;
int enteredPin = 0;

while (enteredPin != correctPin && attempts < 3)
{
    Console.Write("Enter your PIN: ");
    enteredPin = Convert.ToInt32(Console.ReadLine());
    attempts++;
}

if (enteredPin == correctPin)
    Console.WriteLine("Access granted.");
else
    Console.WriteLine("Card blocked. Too many failed attempts.");

```

The `do...while` loop — like `while`, but always runs at least once (condition checked after)

Real-life example — a restaurant menu that shows at least once, then asks if the customer wants to order again:

```

string response;
do
{
    Console.WriteLine("Menu: 1. Rice  2. Matooke  3. Chicken");
    Console.Write("Order placed. Order again? (yes/no): ");
    response = Console.ReadLine();
} while (response.ToLower() == "yes");

Console.WriteLine("Thank you for dining with us!");

```

The `foreach` loop — for iterating over a collection

Real-life example — printing every student name in a class list:

```

string[] classList = { "Nakato", "Okello", "Achen", "Mugisha" };

foreach (string student in classList)
{
    Console.WriteLine($"Present: {student}");
}

```

Explain jump statements in C#

Jump statements transfer control of execution to another part of the program, breaking the normal top-to-bottom flow.

Statement	Purpose	Real-life analogy
<code>break</code>	Immediately exits a loop or <code>switch</code>	A security guard stopping the search line the moment a stolen item is found
<code>continue</code>	Skips the rest of the current iteration, moves to the next one	A cashier skipping an empty till and moving straight to the next customer
<code>return</code>	Exits a method immediately, optionally sending back a value	A messenger who delivers a reply and immediately leaves, without waiting further
<code>goto</code>	Jumps directly to a labelled statement (rarely used; can make code hard to follow)	A shortcut path bypassing the normal route (used sparingly!)

Real-life example — searching a class list for a specific student, stopping once found (`break`):

```

string[] classList = { "Nakato", "Okello", "Achen", "Mugisha" };
string searchName = "Achen";
bool found = false;

foreach (string student in classList)
{
    if (student == searchName)
    {
        found = true;
        break; // no need to keep checking once found
    }
}

Console.WriteLine(found ? $"{searchName} is on the list." : $"{searchName} not found.");

```

Real-life example — skipping absent students when printing attendance (`continue`):

```

string[] attendance = { "Present", "Absent", "Present", "Absent", "Present" };
string[] names = { "Nakato", "Okello", "Achen", "Mugisha", "Tumusiime" };

for (int i = 0; i < names.Length; i++)
{
    if (attendance[i] == "Absent")
        continue; // skip printing this student

    Console.WriteLine($"{names[i]} is present.");
}

```

Try It Yourself

1. Write a program that uses `if...else if...else` to categorise a customer's monthly electricity units into "Lifeline" (0-15 units), "Domestic" (16-100 units), and "Commercial" (above 100 units).
2. Write a `for` loop that prints a multiplication table (1 to 10) for a number entered by the user.
3. Write a program that uses `foreach` to loop through a list of exam scores, using `continue` to skip any score below 50, and printing only the passing scores.

Chapter Summary

- Selection constructs (`if` , `if...else` , `if...else if...else` , `switch`) let a program choose between different execution paths based on conditions.
- `switch` is cleaner than long `if...else if` chains when checking one variable against many discrete values, and C# requires a `break` (or other jump) at the end of each non-empty case.
- Loop constructs (`for` , `while` , `do...while` , `foreach`) repeat a block of code; `for` suits known repeat counts, `while` / `do...while` suit condition-based repetition, and `foreach` suits iterating collections.
- Jump statements (`break` , `continue` , `return` , `goto`) alter the normal flow of control within loops and methods.

Classes and Methods

Real-life analogy: A **class** is like an architect's **blueprint** for a house — it describes what every house built from it will have (rooms, doors, windows) but is not itself a house you can live in. An **object** is an actual house built from that blueprint — you can build many houses (objects) from one blueprint (class), and each one can be painted a different colour (have different field values) while sharing the same overall structure.

Explain classes and objects

A **class** is a user-defined blueprint that combines **data (fields/properties)** and **behaviour (methods)** into a single unit. An **object** is a specific **instance** of a class, created using the `new` keyword.

Real-life example — modelling a `BankAccount` :

```
using System;

class BankAccount
{
    // Fields (data)
    public string AccountHolder;
    public double Balance;

    // Method (behaviour)
    public void Deposit(double amount)
    {
        Balance += amount;
        Console.WriteLine($"Deposited UGX {amount}. New balance: UGX {Balance}");
    }
}

class Program
{
    static void Main()
    {
        // Creating TWO objects (instances) from the SAME class
        BankAccount account1 = new BankAccount();
        account1.AccountHolder = "Nakato Grace";
        account1.Balance = 50000;

        BankAccount account2 = new BankAccount();
        account2.AccountHolder = "Okello James";
        account2.Balance = 120000;

        account1.Deposit(10000);
        account2.Deposit(5000);
    }
}
```

Notice that `account1` and `account2` are independent objects — depositing into `account1` does not affect `account2`'s balance at all, just like depositing money into your bank account doesn't affect your neighbour's account.

Define and describe methods

A **method** is a named block of code that performs a specific task and can be called (invoked) whenever that task is needed — like a specific **service window** at a bank (e.g., "Deposits," "Withdrawals," "Account Opening") that customers can visit whenever they need that particular service.

Method structure:

```
accessModifier returnType MethodName(parameterList)
{
    // method body
    return value; // only if returnType is not "void"
}
```

Real-life example — a method that calculates a taxi fare:

```
static double CalculateFare(double distanceKm, double ratePerKm)
{
    double baseFare = 2000;
    return baseFare + (distanceKm * ratePerKm);
}

static void Main()
{
    double fare = CalculateFare(12, 500);
    Console.WriteLine($"Total fare: UGX {fare}");
}
```

If a method does not need to return anything (e.g., it just prints something), its return type is `void` :

```
static void GreetCustomer(string name)
{
    Console.WriteLine($"Welcome, {name}! How may we assist you today?");
}
```

List the access modifiers

Access modifiers control *where* a class, method, or field can be accessed from — much like the different levels of access in a bank: the **public** counter area (anyone can enter), the **staff-only** back office (only employees), and the **manager's private vault** (only the manager).

Modifier	Accessible from	Real-life analogy
<code>public</code>	Anywhere, no restrictions	The banking hall — open to all customers
<code>private</code>	Only within the same class	The manager's private safe — only the manager (class) can access it
<code>protected</code>	Same class and any derived (child) classes	Family valuables — accessible to the household and its descendants
<code>internal</code>	Anywhere within the same project/assembly	An internal staff memo — visible to all employees of the company, not outsiders
<code>protected internal</code>	Same assembly, OR derived classes in other assemblies	A mix of the "protected" and "internal" rules

Real-life example:

```
class BankAccount
{
    private double balance;           // hidden — only this class can touch it directly
    public string AccountHolder;      // visible to everyone using this class

    public void Deposit(double amount) // public: customers/other code can call this
    {
        balance += amount; // internally uses the private field safely
    }

    public double GetBalance() // a public "window" to safely view the private balance
    {
        return balance;
    }
}
```

This pattern — hiding fields as `private` and exposing safe `public` methods to interact with them — is called **encapsulation**, one of the four pillars of object-oriented programming.

Explain method overloading

Method overloading means defining multiple methods with the **same name** but **different parameter lists** (different number, types, or order of parameters). The compiler decides which version to call based on the arguments you provide — just like a single reception desk that can handle a visitor arriving with just their name, or with their name and appointment time, using the appropriate procedure depending on what information they bring.

Real-life example — a shop's `CalculateTotal` method that works with or without a discount:

```
class Shop
{
    // Version 1: no discount
    public double CalculateTotal(double price, int quantity)
    {
        return price * quantity;
    }

    // Version 2: with a discount percentage
    public double CalculateTotal(double price, int quantity, double discountPercent)
    {
        double total = price * quantity;
        return total - (total * discountPercent / 100);
    }
}

class Program
{
    static void Main()
    {
        Shop shop = new Shop();
        Console.WriteLine(shop.CalculateTotal(5000, 3));           // uses Version 1
        Console.WriteLine(shop.CalculateTotal(5000, 3, 10));       // uses Version 2 (10% off)
    }
}
```

Define and describe constructors and destructors

A **constructor** is a special method, with the same name as the class and no return type, that runs **automatically** when an object is created — used to set up (initialise) the object's initial state. Think of it as the **registration desk** a new student visits on their very first day — it happens automatically as part of “becoming a student,” setting up their initial record.

```
class Student
{
    public string Name;
    public string RegistrationNumber;

    // Constructor
    public Student(string name, string regNumber)
    {
        Name = name;
        RegistrationNumber = regNumber;
        Console.WriteLine($"New student registered: {name} ({regNumber})");
    }
}

class Program
{
    static void Main()
    {
        Student s1 = new Student("Achen Sarah", "UICT/2024/045"); // constructor runs automatically
    }
}
```

A **destructor** (also called a **finalizer**) is a special method that runs when an object is about to be destroyed/garbage-collected, typically used to release unmanaged resources. It uses a tilde (`~`) before the class name and takes no parameters:

```
class Student
{
    ~Student() // destructor
    {
        Console.WriteLine("Student record cleaned up from memory.");
    }
}
```

Note: In modern C#, destructors are rarely written explicitly because the .NET garbage collector handles memory automatically — destructors are mainly used in advanced scenarios involving unmanaged resources (e.g., file handles).

Try It Yourself

1. Create a `Vehicle` class with `private` fields for `PlateNumber` and `FuelLevel`, a constructor that sets both, and public methods `Refuel(double litres)` and `DisplayStatus()`.
2. Add two overloaded methods called `Describe()` to a `Product` class: one that takes just a name, and one that takes a name and a price.
3. Create two `BankAccount` objects using a constructor, deposit different amounts into each, and print both balances to prove they are independent.

Chapter Summary

- A class is a blueprint; an object is a specific instance created from that blueprint using `new`.
- A method is a reusable, named block of code that performs a task, optionally returning a value.
- Access modifiers (`public`, `private`, `protected`, `internal`, `protected internal`) control visibility and enable encapsulation.
- Method overloading allows multiple methods to share a name but differ in their parameter lists.
- Constructors automatically initialise a new object's state; destructors run when an object is being cleaned up, though they are rarely needed explicitly in modern C#.

Inheritance and Polymorphism

Real-life analogy: Think of a family. A **child inherits** general traits from a **parent** (surname, eye colour, land) but can also have their own unique traits and can express even shared traits differently (two children might both “cook,” but one specialises in local dishes and the other in continental dishes). This is exactly how inheritance and polymorphism work in C#.

Define and describe inheritance

Inheritance allows a new class (called the **derived class** or **subclass**) to acquire the fields and methods of an existing class (called the **base class** or **superclass**), promoting code reuse. In C#, inheritance is expressed with a colon (:).

Real-life example — a general `Employee` class, and a more specific `Manager` class that inherits from it:

```
using System;

class Employee
{
    public string Name;
    public double BasicSalary;

    public void Work()
    {
        Console.WriteLine($"{Name} is working.");
    }
}

class Manager : Employee // Manager INHERITS from Employee
{
    public double Allowance; // extra field unique to Manager

    public void ApproveLeave()
    {
        Console.WriteLine($"{Name} approved a leave request."); // uses inherited "Name"
    }
}

class Program
{
    static void Main()
    {
        Manager mgr = new Manager();
        mgr.Name = "Tumusiime Grace"; // inherited field
        mgr.BasicSalary = 2000000; // inherited field
        mgr.Allowance = 500000; // Manager's own field

        mgr.Work(); // inherited method
        mgr.ApproveLeave(); // Manager's own method
    }
}
```

Every `Manager` **is an** `Employee` (plus more) — this “is-a” relationship is the hallmark of good inheritance design. Note that C# only supports **single inheritance** for classes (a class can inherit from only one base class), unlike some languages that allow multiple inheritance.

Explain method overriding

Method overriding allows a derived class to provide its **own specific implementation** of a method that is already defined in its base class, replacing the inherited behaviour. To enable this, the base method must be marked `virtual`, and the derived method must use `override`.

Real-life example — different employee types calculate salary differently:

```

class Employee
{
    public string Name;

    public virtual double CalculateSalary() // "virtual" = can be overridden
    {
        return 500000; // default base salary
    }
}

class SalesExecutive : Employee
{
    public double CommissionEarned;

    public override double CalculateSalary() // provides SPECIFIC behaviour
    {
        return 500000 + CommissionEarned; // base salary + commission
    }
}

class Program
{
    static void Main()
    {
        Employee generic = new Employee();
        SalesExecutive rep = new SalesExecutive { Name = "Achen", CommissionEarned = 150000 };

        Console.WriteLine($"Generic salary: {generic.CalculateSalary()}");
        Console.WriteLine($"Sales rep salary: {rep.CalculateSalary()}");
    }
}

```

Just as every bank teller “processes a transaction,” but a **loans officer** processes it very differently from a **cashier**, overriding lets each subclass customise shared behaviour to fit its own needs.

Define and describe sealed classes

A **sealed class** is a class that **cannot be inherited from** — it marks the “end of the line” for that particular branch of the family tree. You seal a class using the `sealed` keyword, typically to protect its design or for performance reasons.

Real-life example — a `FinalGradeSheet` class that should never be extended, to preserve academic integrity:

```

sealed class FinalGradeSheet
{
    public string StudentName;
    public double FinalScore;

    public void Publish()
    {
        Console.WriteLine($"{StudentName}: FINAL score = {FinalScore} (locked)");
    }
}

// The following would cause a COMPILE ERROR, because FinalGradeSheet is sealed:
// class AppealedGradeSheet : FinalGradeSheet { }

```

You can also mark an individual overridden method as `sealed` to prevent it from being overridden any further down the inheritance chain:

```

class Base { public virtual void Process() { } }
class Middle : Base { public sealed override void Process() { } } // no further overriding allowed
// class Bottom : Middle { public override void Process() { } } // COMPILE ERROR

```

Explain polymorphism

Polymorphism (Greek for “many forms”) means that objects of different derived classes can be treated through a common base class type, while each still behaves according to its own specific (overridden) implementation. This is one of the most powerful ideas in object-oriented programming.

Real-life example — a payroll system that processes many kinds of employees uniformly:

```
using System;
using System.Collections.Generic;

class Employee
{
    public string Name;
    public virtual double CalculateSalary() => 500000;
}

class SalesExecutive : Employee
{
    public double Commission;
    public override double CalculateSalary() => 500000 + Commission;
}

class Manager : Employee
{
    public double Allowance;
    public override double CalculateSalary() => 800000 + Allowance;
}

class Program
{
    static void Main()
    {
        // A single list can hold MANY DIFFERENT employee types,
        // because they all "ARE" Employees (inheritance)
        List<Employee> staff = new List<Employee>
        {
            new Employee { Name = "Okello" },
            new SalesExecutive { Name = "Achen", Commission = 150000 },
            new Manager { Name = "Tumusiime", Allowance = 500000 }
        };

        // POLYMORPHISM: the same line of code calls a DIFFERENT
        // CalculateSalary() implementation depending on the actual object type
        foreach (Employee e in staff)
        {
            Console.WriteLine($"{e.Name}: UGX {e.CalculateSalary()}");
        }
    }
}
```

Output:

```
Okello: UGX 500000
Achen: UGX 650000
Tumusiime: UGX 1300000
```

This is exactly how a real payroll office works: **the same “run payroll” process** is applied to every staff member, but each person’s pay is calculated according to their own specific role — the office doesn’t need a separate, hardcoded procedure for every individual employee.

Try It Yourself

1. Create a base class `Animal` with a `virtual` method `MakeSound()`. Create two derived classes, `Dog` and `Cat`, each overriding `MakeSound()` with their own sound. Store them in a `List<Animal>` and loop through it, calling `MakeSound()` on each.
2. Create a `sealed` class `Certificate` that cannot be inherited from, and confirm (by trying, and seeing the compiler error) that attempting to inherit from it fails.
3. Extend the payroll example above by adding a third employee type, `Intern`, with its own `CalculateSalary()` override, and add it to the `staff` list.

Chapter Summary

- Inheritance lets a derived class reuse and extend the fields and methods of a base class, expressed with `:` in C#, and models “is-a” relationships.
- Method overriding lets a derived class replace a base class’s `virtual` method with its own implementation, using the `override` keyword.
- A `sealed` class cannot be inherited from; a `sealed override` method cannot be overridden further.
- Polymorphism allows objects of different derived types to be treated uniformly through their common base type, while each still executes its own specific overridden behaviour — a cornerstone of flexible, maintainable object-oriented design.

Abstract Classes and Interfaces

Real-life analogy: Think of a **job description template** used by a company's HR department. It says every employee "must be able to submit a report" and "must be able to attend meetings," without saying exactly *how* each person does those things — a sales executive submits a sales report, an accountant submits a financial report. An **abstract class** is like a partially filled template (some things defined, some left blank to be filled in), while an **interface** is like a pure checklist of *promises* with nothing filled in at all.

Define and describe abstract classes

An **abstract class** is a class that **cannot be instantiated directly** (you cannot create an object from it using `new`) and is meant to be used only as a base class for other classes. It can contain a mix of:

- **Abstract methods** — declared but with **no body/implementation**; derived classes *must* provide one.
- **Concrete (regular) methods** — fully implemented and inherited as-is, or optionally overridden.

Real-life example — a general `Shape` that every specific shape must be able to calculate the area of:

```
using System;

abstract class Shape
{
    public string Name;

    // Abstract method: NO body – every derived class MUST implement this
    public abstract double CalculateArea();

    // Concrete method: shared by all shapes, no need to override
    public void DisplayInfo()
    {
        Console.WriteLine($"{Name}: Area = {CalculateArea():F2}");
    }
}

class Circle : Shape
{
    public double Radius;

    public override double CalculateArea() // MUST implement, or compile error
    {
        return Math.PI * Radius * Radius;
    }
}

class Rectangle : Shape
{
    public double Length, Width;

    public override double CalculateArea()
    {
        return Length * Width;
    }
}

class Program
{
    static void Main()
    {
        // Shape shape = new Shape(); // COMPILE ERROR – cannot instantiate an abstract class

        Circle c = new Circle { Name = "Coin", Radius = 3 };
        Rectangle r = new Rectangle { Name = "Classroom Floor", Length = 10, Width = 6 };

        c.DisplayInfo();
        r.DisplayInfo();
    }
}
```

Abstract classes are ideal when related classes share **some** common structure/behaviour, but each must implement certain operations in its own specific way — just as every UICT department “must submit an annual report,” but the *content* of that report differs completely by department.

Explain interfaces

An **interface** is a completely abstract contract: it defines **what** a class must be able to do (method and property signatures), but provides **no implementation at all**. Any class that implements an interface must provide its own concrete implementation of every member declared in it. In C#, interface names conventionally begin with a capital **I**.

Real-life example — a `IPayable` interface that any “payable” entity in a school finance system must implement:

```
using System;

interface IPayable
{
    double CalculatePayment(); // no body – just a signature/contract
}

class Staff : IPayable
{
    public string Name;
    public double MonthlySalary;

    public double CalculatePayment() // must implement, since promised by IPayable
    {
        return MonthlySalary;
    }
}

class Supplier : IPayable
{
    public string CompanyName;
    public double InvoiceAmount;

    public double CalculatePayment()
    {
        return InvoiceAmount;
    }
}

class Program
{
    static void Main()
    {
        IPayable[] payables = {
            new Staff { Name = "Nakato", MonthlySalary = 1200000 },
            new Supplier { CompanyName = "ABC Stationery", InvoiceAmount = 350000 }
        };

        double totalToPay = 0;
        foreach (IPayable p in payables)
        {
            totalToPay += p.CalculatePayment(); // works uniformly, regardless of actual type
        }

        Console.WriteLine($"Total to be paid this month: UGX {totalToPay}");
    }
}
```

Unlike classes (which support only single inheritance), a C# class **can implement multiple interfaces**, since interfaces only describe *contracts*, not shared implementation — like an employee who can simultaneously satisfy several separate job requirements (“must be punctual,” “must be a good communicator,” “must know Excel”).

```

interface IPayable { double CalculatePayment(); }
interface IReportable { string GenerateReport(); }

class Staff : IPayable, IReportable // implementing MULTIPLE interfaces
{
    public double CalculatePayment() => 1200000;
    public string GenerateReport() => "Monthly attendance report generated.";
}

```

Compare abstract classes and interfaces

Aspect	Abstract Class	Interface
Instantiable?	No (cannot use <code>new</code> directly)	No
Can contain implemented (concrete) methods?	Yes	Only default implementations in modern C# (rare for beginners)
Can contain fields?	Yes	No (only properties, methods, events, indexers)
Inheritance	A class can inherit from only one abstract class	A class can implement many interfaces
Access modifiers on members	Allowed (<code>public</code> , <code>protected</code> , etc.)	Members are implicitly <code>public</code>
Real-life analogy	A partially filled job template — some parts already defined	A pure checklist of promises — nothing pre-filled
Best used when	Related classes share significant common code/state	Unrelated classes need to guarantee they support the same capability

Real-life example combining both — a UICT library system:

```

using System;

interface IBorrowable
{
    void Borrow(string borrowerName);
}

abstract class LibraryItem
{
    public string Title;
    public abstract string GetItemType(); // must be implemented by each item type

    public void DisplayTitle()
    {
        Console.WriteLine($"{GetType()} {Title}");
    }
}

class Book : LibraryItem, IBorrowable
{
    public override string GetItemType() => "Book";

    public void Borrow(string borrowerName)
    {
        Console.WriteLine($"{borrowerName} borrowed the book: {Title}");
    }
}

class Program
{
    static void Main()
    {
        Book b = new Book { Title = "C# 4.0: The Complete Reference" };
        b.DisplayTitle();
        b.Borrow("Okello James");
    }
}

```

Here, `LibraryItem` (an abstract class) provides shared structure for all library items, while `IBorrowable` (an interface) is a separate, optional capability that only borrowable items need to promise — a reference-only encyclopedia, for example, might be a `LibraryItem` but **not** `IBorrowable`.

Try It Yourself

1. Create an abstract class `Vehicle` with an abstract method `CalculateFuelCost(double distanceKm)` and a concrete method `DisplayVehicleInfo()`. Create two subclasses, `Car` and `Motorcycle`, each with its own fuel cost calculation.
2. Create an interface `IPrintable` with a method `Print()`. Implement it in two unrelated classes, e.g. `Invoice` and `StudentReport`.
3. In your own words, write a short paragraph explaining when you would choose an abstract class over an interface, using a real-life example of your own (not from this book).

Chapter Summary

- An abstract class cannot be instantiated and may mix abstract (unimplemented) and concrete (implemented) members; it is used when related classes share significant common structure.
- An interface is a pure contract containing no implementation (in its simplest, beginner-level form); a class can implement multiple interfaces, unlike single class inheritance.
- Abstract classes suit closely related class hierarchies; interfaces suit guaranteeing a capability across otherwise unrelated classes.
- Both enable polymorphism: code written against the abstract class or interface type can work uniformly with any concrete class that fulfils it.

Properties and Indexers

Real-life analogy: A **property** is like a **bank teller window** for a private vault: customers never touch the vault (private field) directly — they interact through the teller (the property), who can enforce rules (e.g., “you cannot withdraw a negative amount”). An **indexer** is like giving your class its own “shelf numbering system,” letting you access items inside it using square brackets, [], just like an array.

Define properties in C#

A **property** is a class member that provides a controlled way to read and/or write the value of a private field, using special `get` and `set` blocks (called **accessors**). Properties look like fields from the outside (no parentheses when used) but behave like methods internally, allowing validation logic.

Real-life example — a `BankAccount` with a protected `Balance` property that prevents a negative balance:

```
using System;

class BankAccount
{
    private double balance; // private field – hidden from outside

    public double Balance    // public property – the "teller window"
    {
        get { return balance; }
        set
        {
            if (value < 0)
            {
                Console.WriteLine("Error: Balance cannot be negative.");
            }
            else
            {
                balance = value;
            }
        }
    }
}

class Program
{
    static void Main()
    {
        BankAccount acc = new BankAccount();
        acc.Balance = 50000;    // calls the "set" accessor
        Console.WriteLine(acc.Balance); // calls the "get" accessor

        acc.Balance = -1000;    // rejected by validation logic inside "set"
    }
}
```

Auto-implemented properties

When no extra validation logic is needed, C# lets you write a shorter **auto-implemented property**, where the compiler creates the hidden private field for you automatically:

```
class Student
{
    public string Name { get; set; }           // auto-implemented property
    public string RegistrationNumber { get; set; }
    public int Age { get; private set; }       // can be read publicly, but only set from inside the class
}
```

Explain properties, fields, and methods

It is important for a beginner to clearly distinguish these three related but different concepts:

Concept	What it is	Real-life analogy
Field	A raw variable that stores data directly inside an object	The actual cash sitting inside the bank vault
Property	A controlled “gateway” (get/set) to read or modify a field’s value, often with validation	The teller window through which customers interact with the vault
Method	A named block of code that performs an action or computation, which may or may not involve fields	A specific banking service, e.g. “Transfer Funds,” “Print Statement”

Real-life example showing all three together:

```
class Student
{
    private double totalMarks; // FIELD – raw data storage

    public double TotalMarks // PROPERTY – controlled access to the field
    {
        get { return totalMarks; }
        set { totalMarks = (value < 0) ? 0 : value; } // never allow negative marks
    }

    public string GetGrade() // METHOD – performs a calculation/action
    {
        if (TotalMarks >= 80) return "A";
        else if (TotalMarks >= 60) return "B";
        else return "C";
    }
}
```

Best practice: In modern C#, fields are almost always kept `private`, and outside code interacts with the object only through `public` properties and methods — this is the practical application of **encapsulation**.

Explain indexers

An **indexer** allows objects of a class to be accessed using array-like square bracket syntax (`object[index]`), even though the class is not actually an array. It is defined using the `this` keyword.

Real-life example — a `ClassRegister` that lets you look up a student’s name by their seat number, just like an array:

```
using System;

class ClassRegister
{
    private string[] students = new string[40]; // internal storage (private!)

    // Indexer definition
    public string this[int seatNumber]
    {
        get { return students[seatNumber]; }
        set { students[seatNumber] = value; }
    }
}

class Program
{
    static void Main()
    {
        ClassRegister register = new ClassRegister();

        register[0] = "Nakato Grace"; // uses the indexer's "set"
        register[1] = "Okello James";

        Console.WriteLine(register[0]); // uses the indexer's "get" – prints "Nakato Grace"
    }
}
```

Indexers are especially useful when your class is essentially a **custom collection** or **wrapper around data**, and you want it to feel as natural to use as a built-in array or list — for example, a `Dictionary<string, double>`-like class for looking up product prices by product code:

```
class PriceList
{
    private System.Collections.Generic.Dictionary<string, double> prices = new();

    public double this[string productCode]
    {
        get => prices.ContainsKey(productCode) ? prices[productCode] : 0;
        set => prices[productCode] = value;
    }
}

// Usage:
PriceList shopPrices = new PriceList();
shopPrices["SUGAR1KG"] = 4500;
Console.WriteLine(shopPrices["SUGAR1KG"]); // 4500
```

Try It Yourself

1. Create a `Product` class with a private field `stockQuantity` and a public property `StockQuantity` that prevents the value from ever being set below zero.
2. Convert a class of your choice (e.g., `Student`) to use auto-implemented properties instead of manually written fields and `get/set` blocks.
3. Create a `Library` class with an indexer that allows looking up book titles by an integer shelf position, similar to the `ClassRegister` example.

Chapter Summary

- A property provides controlled, validated access to a private field, using `get` and `set` accessors, while looking like a simple field from the outside.
- Auto-implemented properties (`{ get; set; }`) offer a shorter syntax when no custom validation logic is needed.
- Fields store raw data, properties control access to that data, and methods perform actions — together they form the building blocks of a well-encapsulated class.
- Indexers allow objects to be accessed using array-like square bracket syntax, defined with the `this` keyword, and are ideal for custom collection-like classes.

Generics and Iterators

Real-life analogy: A **generic** class is like a **standard shipping container** — the container's design (loading, stacking, locking mechanism) works the same way whether it's carrying rice, furniture, or electronics; you don't need a completely different container design for every type of cargo. An **iterator** is like a **tour guide** who walks a group through a museum room by room, handing out each exhibit's description one at a time, without needing the entire tour pre-planned and stored in memory at once.

Define and describe generics

Generics allow you to design classes, methods, and interfaces with a **placeholder type** (commonly written as `T`) that is specified later, when the class or method is actually used. This lets you write **one** reusable, type-safe implementation instead of duplicating code for every data type.

Why generics matter — the problem they solve:

Without generics, you might need to write nearly identical classes for every data type you want to store:

```
class IntBox { public int Value; }
class StringBox { public string Value; }
class StudentBox { public Student Value; }
// ...repetitive and hard to maintain
```

With generics, one class handles **all** of these cases:

```
class Box<T> // T is a placeholder for "some type, to be decided later"
{
    public T Value;

    public void Display()
    {
        Console.WriteLine($"Box contains: {Value}");
    }
}
```

Explain creating and using generics

Real-life example — a generic `Store<T>` class that can safely hold any single type of item, such as a warehouse pallet dedicated to just one product line:

```

using System;

class Store<T>
{
    private T item;

    public void AddItem(T newItem)
    {
        item = newItem;
        Console.WriteLine($"Stored: {item}");
    }

    public T GetItem()
    {
        return item;
    }
}

class Program
{
    static void Main()
    {
        Store<string> nameStore = new Store<string>(); // T becomes "string" here
        nameStore.AddItem("Nakato Grace");

        Store<int> ageStore = new Store<int>(); // T becomes "int" here – SAME class, different type!
        ageStore.AddItem(20);

        // Store<int> wrongStore = new Store<int>();
        // wrongStore.AddItem("hello"); // COMPILE ERROR – type safety enforced!
    }
}

```

C#'s built-in `List<T>` and `Dictionary<TKey, TValue>` (from `System.Collections.Generic`) are themselves generic classes, and are the most commonly used generics in everyday C# programming:

```

using System.Collections.Generic;

List<string> studentNames = new List<string>(); // a generic, type-safe, growable list
studentNames.Add("Nakato");
studentNames.Add("Okello");

Dictionary<string, double> productPrices = new Dictionary<string, double>();
productPrices["Sugar"] = 4500;
productPrices["Soap"] = 3000;

```

Real-life example — a generic method that finds the maximum of any two comparable values, e.g., comparing exam scores or product prices with the same logic:

```

static T FindMax<T>(T a, T b) where T : IComparable<T>
{
    return a.CompareTo(b) > 0 ? a : b;
}

// Usage:
Console.WriteLine(FindMax<int>(45, 78));           // works with numbers
Console.WriteLine(FindMax<double>(4500.5, 3200.25)); // works with decimals
Console.WriteLine(FindMax<string>("Achen", "Okello")); // works with text (alphabetical order)

```

The `where T : IComparable<T>` is a **generic constraint** — it tells the compiler “T can be any type, as long as it supports comparison,” preventing misuse with types that don’t make sense to compare.

Explain iterators

An **iterator** is a method that allows you to step through (iterate over) a sequence of values **one at a time**, without needing to build and store the entire sequence in memory first. In C#, iterators are implemented using the `yield return` keyword.

Real-life analogy: Imagine an examiner marking scripts one by one and calling out each result as soon as it's ready, rather than waiting to mark every single script in the entire pile before announcing any results. This is exactly what `yield return` does — it produces results **on demand**, one at a time.

Real-life example — an iterator that generates a list of even-numbered admission slots without pre-building a huge array:

```
using System;
using System.Collections.Generic;

class Program
{
    static IEnumerable<int> GetEvenSlots(int maxSlots)
    {
        for (int i = 2; i <= maxSlots; i += 2)
        {
            yield return i; // produces ONE value, then pauses here until asked for the next one
        }
    }

    static void Main()
    {
        foreach (int slot in GetEvenSlots(10))
        {
            Console.WriteLine($"Slot number: {slot}");
        }
    }
}
```

Output:

```
Slot number: 2
Slot number: 4
Slot number: 6
Slot number: 8
Slot number: 10
```

Real-life example — an iterator over a custom `ClassRoster` class, so it can be used naturally in a `foreach` loop like any built-in collection:

```

using System;
using System.Collections;
using System.Collections.Generic;

class ClassRoster : IEnumerable<string>
{
    private List<string> students = new List<string> { "Nakato", "Okello", "Achen" };

    public IEnumerator<string> GetEnumerator()
    {
        foreach (string student in students)
        {
            yield return student; // hand out one student at a time
        }
    }

    IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
}

class Program
{
    static void Main()
    {
        ClassRoster roster = new ClassRoster();

        foreach (string name in roster) // works naturally because of the iterator above
        {
            Console.WriteLine($"Calling register: {name}");
        }
    }
}

```

Why iterators matter: they are especially valuable for large or even infinite sequences (e.g., reading a huge log file line by line), because values are produced lazily — only when actually needed — instead of consuming large amounts of memory up front.

Try It Yourself

1. Create a generic class `Pair<T>` that stores two values of the same type and has a method `Swap()` that swaps them.
2. Write a generic method `PrintAll<T>(List<T> items)` that prints every item in any list, regardless of type, and test it with a `List<string>` and a `List<int>`.
3. Write an iterator method `GetWeekdays()` using `yield return` that produces the names of the five weekdays (Monday to Friday) one at a time, and print them using `foreach`.

Chapter Summary

- Generics let you write a single, reusable, type-safe class or method using a placeholder type `T`, avoiding the need to duplicate code for every data type — `List<T>` and `Dictionary<TKey, TValue>` are the most commonly used built-in generics.
- Generic constraints (e.g., `where T : IComparable<T>`) restrict what types `T` can be, ensuring the generic code makes sense for any type actually used.
- Iterators, implemented with `yield return`, produce a sequence of values one at a time, on demand, without needing the whole sequence stored in memory at once.
- Implementing `IEnumerable<T>` with an iterator lets your own custom classes be used naturally in `foreach` loops, just like built-in collections.

Exception Handling

Real-life analogy: Think of a **fire alarm and evacuation plan** in a building. Normal business goes on (the main program flow) until something goes wrong (a fire — an exception). Instead of the whole building collapsing in chaos, there is a planned procedure: sound the alarm (`throw`), guide people to safety (`catch`), and always perform certain final tasks such as locking the doors after everyone is out (`finally`) — regardless of which particular emergency occurred.

Define and describe exceptions

An **exception** is an unexpected or erroneous event that occurs while a program is running, disrupting its normal flow of execution — for example, trying to divide by zero, accessing an item that doesn't exist, or trying to open a file that was deleted. If left unhandled, an exception causes the program to crash.

Example of an unhandled exception:

```
using System;

class Program
{
    static void Main()
    {
        int totalStudents = 30;
        int groups = 0;

        int studentsPerGroup = totalStudents / groups; // ERROR: division by zero!
        Console.WriteLine(studentsPerGroup); // this line is never reached
    }
}
```

Running this program crashes with an **unhandled exception**:

```
Unhandled Exception: System.DivideByZeroException: Attempted to divide by zero.
```

Exception handling is the process of anticipating such problems and dealing with them gracefully — showing a helpful message to the user and allowing the rest of the program (or at least the application) to continue running safely, instead of crashing outright.

Explain the process of throwing and catching exceptions

C# uses four keywords for exception handling: `try`, `catch`, `finally`, and `throw`.

- **`try`** — wraps a block of code that *might* cause an exception.
- **`catch`** — defines what to do *if* a specific exception occurs inside the `try` block.
- **`finally`** — a block that *always* runs, whether or not an exception occurred (used for clean-up, e.g., closing a file or database connection).
- **`throw`** — manually raises (creates) an exception.

Real-life example — safely handling a division at an examination results system, avoiding a crash if the number of groups is zero:

```

using System;

class Program
{
    static void Main()
    {
        int totalStudents = 30;
        int groups = 0;

        try
        {
            int studentsPerGroup = totalStudents / groups;
            Console.WriteLine($"Students per group: {studentsPerGroup}");
        }
        catch (DivideByZeroException ex)
        {
            Console.WriteLine("Error: Number of groups cannot be zero.");
            Console.WriteLine($"Details: {ex.Message}");
        }
        finally
        {
            Console.WriteLine("Group allocation process finished.");
        }

        Console.WriteLine("Program continues running normally...");
    }
}

```

Output:

```

Error: Number of groups cannot be zero.
Details: Attempted to divide by zero.
Group allocation process finished.
Program continues running normally...

```

Notice how, unlike the earlier unhandled crash, the program **recovers** and keeps running.

Real-life example — manually throwing an exception when invalid data is entered, such as a negative deposit amount at a bank:

```

class BankAccount
{
    public double Balance;

    public void Deposit(double amount)
    {
        if (amount < 0)
        {
            throw new ArgumentException("Deposit amount cannot be negative."); // manually "raise the alarm"
        }
        Balance += amount;
    }
}

class Program
{
    static void Main()
    {
        BankAccount acc = new BankAccount();
        try
        {
            acc.Deposit(-5000);
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine($"Transaction rejected: {ex.Message}");
        }
    }
}

```

Explain nested try and multiple catch blocks

Multiple catch blocks

A single `try` block can be followed by **several** `catch` blocks, each handling a **different, specific type of exception**. C# checks them in order, from the most specific exception type to the most general, and executes the first one that matches.

Real-life example — a school records lookup system that can fail in different ways:

```

using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        Dictionary<string, int> examScores = new Dictionary<string, int>
        {
            { "UICT/2024/001", 85 },
            { "UICT/2024/002", 72 }
        };

        Console.Write("Enter registration number: ");
        string regNo = Console.ReadLine();

        try
        {
            int score = examScores[regNo]; // may throw KeyNotFoundException
            int groups = Convert.ToInt32(Console.ReadLine()); // may throw FormatException
            int average = score / groups; // may throw DivideByZeroException

            Console.WriteLine($"Result: {average}");
        }
        catch (KeyNotFoundException)
        {
            Console.WriteLine("Error: Registration number not found.");
        }
        catch (FormatException)
        {
            Console.WriteLine("Error: Please enter a valid whole number for groups.");
        }
        catch (DivideByZeroException)
        {
            Console.WriteLine("Error: Number of groups cannot be zero.");
        }
        catch (Exception ex) // general "catch-all" — always placed LAST
        {
            Console.WriteLine($"An unexpected error occurred: {ex.Message}");
        }
    }
}

```

Important rule: the general `catch (Exception ex)` block must always be placed **last**, since C# matches catch blocks from top to bottom, and a general exception type would otherwise “steal” and hide more specific errors that should have been caught by their own dedicated block.

Nested try blocks

A `try` block can contain **another try block** inside it — useful when a smaller, specific operation within a larger process needs its own dedicated error handling.

Real-life example — processing a whole batch of admissions, where one bad record should not stop the entire batch:

```

using System;

class Program
{
    static void Main()
    {
        string[] applicantAges = { "18", "abc", "21", "17" }; // one invalid entry

        try // OUTER try: handles problems with the whole batch process
        {
            foreach (string ageText in applicantAges)
            {
                try // INNER try: handles problems with just ONE record
                {
                    int age = Convert.ToInt32(ageText);
                    Console.WriteLine(age >= 18
                        ? $"Applicant (age {age}) is eligible."
                        : $"Applicant (age {age}) is under-age.");
                }
                catch (FormatException)
                {
                    Console.WriteLine($"Skipped invalid record: '{ageText}'");
                    continue; // move on to the next applicant, batch is not stopped
                }
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Batch processing failed entirely: {ex.Message}");
        }

        Console.WriteLine("Batch processing complete.");
    }
}

```

Define and describe custom exceptions

Sometimes the built-in exception types (`DivideByZeroException`, `FormatException`, `ArgumentException`, etc.) don't precisely describe a problem specific to your own application's rules. C# lets you create **custom exceptions** by defining a class that inherits from `Exception` (or a more specific existing exception type).

Real-life example — a custom `InsufficientFundsException` for a banking application, which communicates the exact business rule that was broken far more clearly than a generic error would:

```

using System;

// Custom exception class
class InsufficientFundsException : Exception
{
    public InsufficientFundsException(string message) : base(message)
    {
    }
}

class BankAccount
{
    public double Balance;

    public void Withdraw(double amount)
    {
        if (amount > Balance)
        {
            throw new InsufficientFundsException(
                $"Cannot withdraw UGX {amount}. Available balance is only UGX {Balance}.");
        }
        Balance -= amount;
        Console.WriteLine($"Withdrawal successful. New balance: UGX {Balance}");
    }
}

class Program
{
    static void Main()
    {
        BankAccount acc = new BankAccount { Balance = 20000 };

        try
        {
            acc.Withdraw(50000);
        }
        catch (InsufficientFundsException ex)
        {
            Console.WriteLine($"Transaction declined: {ex.Message}");
        }
    }
}

```

Custom exceptions make your code far more **readable and maintainable** — a `catch (InsufficientFundsException ex)` block immediately tells any programmer reading the code exactly what business rule failed, far more clearly than a generic `catch (Exception ex)` ever could.

Try It Yourself

1. Write a program that asks the user to enter two numbers and divides them, using `try/catch` to handle both a `DivideByZeroException` and a `FormatException` (if the user types letters instead of numbers).
2. Create a custom exception `InvalidAgeException` and use it inside a `RegisterVoter(int age)` method that throws it if the age is below 18.
3. Write a program with a nested `try` block that processes a list of five product prices (as strings), skipping and reporting any invalid entries without stopping the rest of the batch, similar to the admissions example above.

Chapter Summary

- An exception is an unexpected event that disrupts a program's normal flow; unhandled exceptions cause the program to crash.
- `try` wraps risky code, `catch` handles a specific exception type if it occurs, `finally` always runs regardless of the outcome, and `throw` manually raises an exception.
- Multiple `catch` blocks can handle different exception types from the same `try` block, checked in order, with the general `catch (Exception ex)` always placed last.
- Nested `try` blocks allow fine-grained error handling for a specific sub-operation within a larger process, without stopping the entire operation.
- Custom exceptions, created by inheriting from `Exception`, let you define application-specific error types that make your code's error handling clearer and more meaningful.

Conclusion

Congratulations on completing this journey through C# programming! You have moved from the basic building blocks of variables, operators, and control structures, through the pillars of object-oriented programming — classes, inheritance, polymorphism, abstraction — all the way to advanced features like generics, iterators, and robust exception handling.

C# and the .NET platform power a huge range of real-world software: banking systems, hospital management systems, e-commerce platforms, mobile apps, and enterprise software used across Uganda and the world. Everything you have learned in this book forms the essential foundation for the next stages of your studies, including ADO.NET and SQL Server for database-driven applications, web development with ASP.NET, and mobile development.

Keep practicing. As I always tell my students at UICT: programming is a skill of the hands as much as the mind. Open Visual Studio every day, build small real-life projects — a class register, a simple banking simulator, a school fees calculator — and you will be more than ready to pass this course highly and confidently apply these skills in your career.

— *Jam Savage, UICT*